

Credit Card Fraud Detection

Machine Learning Mini Lab — All 10 Experiments

Complete Machine Learning Development Life Cycle (MLDLC)

Project Report

May 2026

■ Table of Contents

1. Executive Summary
2. Project Overview & Dataset
3. Experiment 1 — Problem Definition
4. Experiment 2 — Dataset Understanding & Structure
5. Experiment 3 — Problem Type Identification
6. Experiment 4 — Data Cleaning
7. Experiment 5 — Data Preprocessing
8. Experiment 6 — Exploratory Data Analysis (EDA)
9. Experiment 7 — Feature Selection & Engineering
10. Experiment 8 — Model Training & Selection
11. Experiment 9 — Model Evaluation & Tuning
12. Experiment 10 — Model Deployment
13. Final Summary & Conclusions

1. Executive Summary

This project presents a complete implementation of the Machine Learning Development Life Cycle (MLDLC) applied to the problem of Credit Card Fraud Detection. The notebook covers all ten key experiments — from problem definition through to model deployment — using a synthetic dataset of 1,000 credit card transactions.

Best Model Result: Tuned Random Forest (GridSearchCV) achieved **99.5% Accuracy**, **97.96% Recall**, **100% Precision**, **F1-Score of 0.9897**, and **ROC-AUC of 0.9998**. Only 2 fraudulent transactions were missed out of 98 in the test set.

Metric	Logistic Regression	Decision Tree	Random Forest (Tuned)
Accuracy	81.00%	99.50%	99.00%
Recall	83.67%	98.98%	97.96%
Precision	78.85%	100.0%	100.0%
F1-Score	81.19%	99.49%	98.97%
ROC-AUC	81.05%	99.49%	99.98%

Table 1: Final model comparison across all evaluation metrics.

2. Project Overview & Dataset

Credit card fraud is a major financial threat. Banks and payment companies need to automatically identify whether a transaction is fraudulent or legitimate to protect customers. This project trains and evaluates multiple ML classifiers on labelled transaction data.

Dataset at a Glance

Property	Details
Source	Synthetic credit card transactions (Kaggle)
Rows	1,000 transactions
Columns	13 (9 features + 3 identifiers + 1 target)
Target	Fraud — 0 = Legitimate, 1 = Fraudulent
Class split	~51% Legitimate / ~49% Fraudulent (well balanced)
Numeric features	Age, Transaction Amount, International Transaction, Card Present, Failed Attempts
Categorical features	Gender, Merchant Category, City, Transaction Time

Feature Descriptions

Feature	Type	Description
Age	Numeric	Cardholder age (18–94)
Gender	Categorical	Male / Female
Transaction_Amount	Numeric	Amount in local currency
Merchant_Category	Categorical	Food, Travel, Online Shopping, etc.
City	Categorical	Transaction city
Transaction_Time	DateTime	Timestamp of transaction
International_Transaction	Binary (0/1)	Cross-border flag
Card_Present	Binary (0/1)	Physical card used
Failed_Attempts	Numeric	Prior failed transaction attempts
Fraud	Target (0/1)	0 = Legitimate, 1 = Fraudulent

3. Experiment 1 — Problem Definition

Objective

Define the ML problem clearly and understand what we are trying to achieve.

Problem Statement

Credit card fraud is a major financial threat. Banks and payment companies need to automatically identify whether a transaction is fraudulent or legitimate to protect customers.

Item	Details
Real-world problem	Detecting fraudulent credit card transactions
Objective	Build a classifier to predict if a transaction is fraud (1) or not (0)
Input Features (X)	Age, Gender, Transaction Amount, Merchant Category, City, Transaction Time, International flag, Card Present, Failed Attempts
Target Variable (Y)	Fraud — binary: 0 = Legitimate, 1 = Fraudulent
Dataset	Synthetic credit card transactions (1,000 rows, 13 columns)

Why Machine Learning? Manual rule-based systems cannot handle the complexity and volume of modern transactions. ML models learn subtle patterns automatically and generalise to unseen data.

4. Experiment 2 — Dataset Understanding & Structure

Objective

Load the dataset and understand its structure — rows, columns, data types, features vs. target.

Dataset Output (from notebook)

Column	Non-Null Count	Dtype
Transaction_ID	1000	object
Cardholder_Name	1000	object
Age	1000	int64
Gender	1000	object
Transaction_Amount	1000	float64
Merchant_Name	1000	object
Merchant_Category	1000	object
City	1000	object
Transaction_Time	1000	object
International_Transaction	1000	int64
Card_Present	1000	int64
Failed_Attempts	1000	int64
Fraud	1000	int64

Statistical Summary (Key Numeric Columns)

Statistic	Age	Transaction_Amount	Failed_Attempts
Count	1000	1000	1000
Mean	44.21	2488.79	~2.5
Std	14.70	1476.60	~1.6
Min	18	11.19	0
25%	32	1149.68	1
50%	45	2500.40	3
75%	57	3800+	4
Max	~89	~7500	~7

Key Observations: 1,000 rows, 13 columns — manageable size. Nearly balanced: ~49% fraud, ~51% legitimate. Mix of numeric and categorical columns. Transaction_ID, Cardholder_Name, and Merchant_Name are identifiers — not useful as features.

5. Experiment 3 — Problem Type Identification

Objective

Identify the exact type of ML problem we are solving.

Question	Answer	Reason
Supervised vs Unsupervised?	SUPERVISED	Labelled target column 'Fraud' exists
Classification vs Regression?	CLASSIFICATION	Target is categorical (0 or 1)
Binary vs Multi-class?	BINARY CLASSIFICATION	Only 2 classes: fraud or legitimate
Balanced vs Imbalanced?	BALANCED	Fraud rate ~49% — well balanced
Key evaluation metric?	Recall, F1, ROC-AUC	Accuracy alone misleads on fraud detection

Conclusion: Supervised Binary Classification. Algorithms: Logistic Regression, Decision Tree, Random Forest. Priority: High Recall — missing fraud is most costly.

6. Experiment 4 — Data Cleaning

Objective

Handle missing values, duplicates, outliers, and invalid entries.

Check	Result	Action Taken
Missing Values	0 missing cells	No action needed
Duplicate Rows	0 duplicates	No action needed
Outliers (Age)	0 outliers	IQR clipping range: [-5.50, 94.50]
Outliers (Amount)	0 outliers	IQR clipping range: [-2796.27, 7726.26]
Outliers (Failed Attempts)	0 outliers	IQR clipping range: [-3.50, 8.50]
Invalid Values	None found	Logical checks passed
Final shape	1,000 rows x 13 columns	Dataset unchanged

What we did: No missing values or duplicates were found. Outlier capping via the IQR method preserves row count while limiting extreme values. Logical constraint checks (age > 0, amount > 0, etc.) all passed.

7. Experiment 5 — Data Preprocessing

Objective

Encode categories, extract features from datetime, and scale numeric columns.

Step	Action	Output Shape
1. Drop IDs	Removed Transaction_ID, Cardholder_Name, Merchant_Name	(1000, 10)
2. DateTime features	Extracted Hour, Month, DayOfWeek, IsWeekend, IsNight from Transaction_Time	(1000, 14)
3. Binary encode	Gender: Male=0, Female=1	No shape change
4. One-Hot Encode	OHE applied to Merchant_Category & City (drop_first=True)	(1000, 25)
5. Train/Test split	80/20 stratified split (random_state=42)	Train: (800,24) / Test: (200,24)
6. StandardScaler	Scaled: Age, Transaction_Amount, Failed_Attempts, Hour (fit on train only)	No shape change

Key Decisions: Dropped ID columns — no predictive value. DateTime features capture time-based fraud patterns (late-night, weekend). One-Hot Encoding for nominal features with drop_first=True avoids the dummy variable trap. StandardScaler fit on train only — prevents data leakage.

8. Experiment 6 — Exploratory Data Analysis (EDA)

Objective

Visualise patterns to understand what drives fraud.

Key Findings

Finding	Detail
Class balance	~510 Legitimate vs ~490 Fraudulent — nearly balanced dataset
Transaction Amount	Distribution overlaps heavily between fraud & legitimate — not a strong standalone predictor
Merchant Category	
Failed Attempts	Fraud rates differ across merchant categories (varies by type)
International Trans.	Fraud rate increases strongly with number of prior failed attempts
Card Present flag	Higher fraud rate for international transactions
Correlation (top)	Transactions with Card_Present=0 associate more with fraud
	Failed_Attempts: +0.65 Transaction_Amount: +0.33 International: +0.27

Correlation with Fraud (Top Features)

Feature	Correlation with Fraud	Direction
Failed_Attempts	0.648	Positive
Transaction_Amount	0.334	Positive
International_Transaction	0.273	Positive
Merchant_Category_Online Shopping	0.099	Positive
City_Chennai	0.068	Positive
Age	-0.059	Negative
Merchant_Category_Food	-0.051	Negative

Key Insight: Failed_Attempts is the strongest predictor of fraud. International transactions and card-not-present also associate with fraud. Transaction amount alone does not cleanly separate fraud from legitimate transactions.

9. Experiment 7 — Feature Selection & Engineering

Objective

Select the most informative features to reduce noise and improve model performance.

Random Forest Feature Importances (Top 10)

Rank	Feature	Importance Score
1	Failed_Attempts	0.4834
2	Transaction_Amount	0.1702
3	International_Transaction	0.0932
4	Age	0.0436
5	Hour	0.0401
6	Month	0.0323
7	DayOfWeek	0.0261
8	Merchant_Category_Online Shopping	0.0132
9	Gender	0.0094
10	City_Chennai	0.0081

Chi-Square SelectKBest — Selected Features (k=10)

#	Selected Feature
1	Transaction_Amount
2	International_Transaction
3	Failed_Attempts
4	DayOfWeek
5	IsWeekend
6	Merchant_Category_Fashion
7	Merchant_Category_Food
8	Merchant_Category_Online Shopping
9	City_Chennai
10	City_Mumbai

Three methods used: Correlation filtering (linear relationships, train only), Random Forest importance (captures non-linear relationships without leakage), and Chi-Square SelectKBest (statistical test for top-k relevant features). Final selected dataset: X_train (800x10) and X_test (200x10).

10. Experiment 8 — Model Training & Selection

Objective

Train three ML models on the selected features and compare their performance.

Models Trained

Model	Configuration
Logistic Regression	max_iter=2000, class_weight='balanced', random_state=42
Decision Tree	class_weight='balanced', random_state=42
Random Forest	n_estimators=100, class_weight='balanced', random_state=42

Training Results on Test Set (200 samples)

Model	Accuracy	Recall	Precision	F1-Score	ROC-AUC
Logistic Regression	81.00%	83.67%	78.85%	81.19%	81.05%
Decision Tree	99.50%	98.98%	100.0%	99.49%	99.49%
Random Forest	99.50%	98.98%	100.0%	99.49%	~99.49%

Best Model (F1-Score): Decision Tree and Random Forest both achieved 99.49% F1-score. Random Forest was selected for hyperparameter tuning due to its ensemble robustness and lower risk of overfitting.

11. Experiment 9 — Model Evaluation & Tuning

Objective

Evaluate all models with multiple metrics and tune the best model using GridSearchCV.

Confusion Matrix Results

Model	TN	FP	FN	TP	Recall	F1-Score
Logistic Regression	80	22	16	82	83.67%	81.19%
Decision Tree	102	0	1	97	98.98%	99.49%
Random Forest	102	0	1	97	98.98%	99.49%

Guide: TN = correct legit | FP = false alarm | FN = missed fraud (most costly!) | TP = detected fraud

GridSearchCV — Hyperparameter Tuning (Random Forest)

Parameter	Values Tried	Best Value
n_estimators	[50, 100, 150]	100
max_depth	[None, 5, 10]	None
min_samples_split	[2, 5]	2
class_weight	[None, 'balanced']	None
CV folds	3	3-fold cross-validation
Scoring metric	recall	Best CV Recall: 0.98

Tuned Random Forest — Final Performance

Class	Precision	Recall	F1-Score	Support
Legitimate	0.98	1.00	0.99	102
Fraudulent	1.00	0.98	0.99	98
Accuracy			0.99	200
Macro Avg	0.99	0.99	0.99	200
Weighted Avg	0.99	0.99	0.99	200

Key numbers: Fraud Missed (FN) = 2 | Fraud Detected (TP) = 96 | False Alarms (FP) = 0 | Recall = 0.9796 | ROC-AUC = 0.9998

12. Experiment 10 — Model Deployment

Objective

Create a prediction function that takes user input and returns a fraud decision.

predict_fraud() Function Pipeline

Step	Action
1. Receive input	Age, Gender, Transaction Amount, Merchant Category, City, Hour, Month, International flag, Card Present, Failed Attempts
2. Feature engineering	Compute IsWeekend and IsNight from Hour and DayOfWeek
3. Binary encode	Gender: Male=0, Female=1
4. One-Hot encode	Align Merchant_Category and City to training columns
5. Scale numeric cols	Apply pre-fitted StandardScaler (Age, Amount, Failed Attempts, Hour)
6. Predict	Random Forest predict_proba() → fraud probability
7. Apply threshold	Default threshold = 40% — returns FRAUD / LEGITIMATE + risk tier

Sample Predictions

Scenario	Key Inputs	Prediction	Fraud Prob.	Risk Level
High-risk	Age=28, Male, Amount=4500, Online Shopping, Mumbai, Hour=3, Intl=1, Failed=4	FRAUD	100.00%	HIGH
Low-risk	Age=45, Female, Amount=250, Grocery, Delhi, Hour=14, Intl=0, Failed=0	LEGITIMATE	~1%	LOW
Interactive test	Age=24, Male, Amount=2400, Food, Delhi, Hour=19, Intl=0, Failed=1	LEGITIMATE	1.00%	LOW

Deployment artefacts saved: *model.pkl* (tuned Random Forest) and *scaler.pkl* (fitted StandardScaler). The *predict_fraud()* function handles full preprocessing in a single call and is ready for integration into a REST API or web application.

13. Final Summary & Conclusions

Exp.	Topic	Status	Key Outcome
1	Problem Definition	DONE	Supervised Binary Classification defined
2	Dataset Understanding	DONE	1000 rows, 13 cols, ~50% fraud rate
3	Problem Type	DONE	Binary classification confirmed
4	Data Cleaning	DONE	No missing values, no duplicates, IQR capping
5	Data Preprocessing	DONE	OHE, scaling, datetime features, 80/20 split
6	EDA	DONE	Failed_Attempts is strongest fraud signal
7	Feature Selection	DONE	Top 10 features via RF + Chi-Square
8	Model Training	DONE	DT & RF both 99.5% accuracy
9	Evaluation & Tuning	DONE	Tuned RF: Recall=97.96%, ROC-AUC=0.9998
10	Deployment	DONE	predict_fraud() + model.pkl saved

Conclusions

Best Model: Tuned Random Forest (GridSearchCV) — Recall 97.96%, Precision 100%, ROC-AUC 0.9998.

Key Predictor: Failed_Attempts (importance 0.4834) is the single strongest signal of fraud — far above all other features.

Zero False Positives: The tuned model produces no false alarms while missing only 2 out of 98 fraudulent transactions.

Well-Balanced Dataset: The ~50/50 class split simplified modelling — in real-world datasets with severe imbalance, SMOTE or class_weight tuning would be critical.

Production-Ready: The deployment function handles the full preprocessing pipeline in a single call, making integration into production systems straightforward.

Recommended next steps: (1) Test on real-world imbalanced data with SMOTE. (2) Add ROC curve and Precision-Recall curve visualisations. (3) Deploy predict_fraud() as a REST API using FastAPI or Flask. (4) Add model versioning and monitoring for drift detection.